

Coding with Claude Code

Yale Library | StatLab

2026-06-09

Workshop Flow

Segment	Time	Goal
Crash course	20 min	Learn the mental model for Claude Code
Follow-along demo	30 min	Use Claude Code on a real project folder
Q&A	10 min	Connect the workflow to your own work

What You Should Be Able To Do

By the end, you should be able to:

1. Explain how Claude Code is different from a chat assistant.
2. Open Claude Code in the right working folder.
3. Write a prompt with a clear task, files, output, and done condition.
4. Review Claude's actions before trusting the result.
5. Know when to use Plan Mode, `/clear`, and project instructions.

The Big Idea

Claude Code is a coding agent.

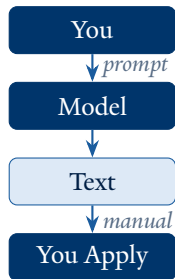
That means it can:

- Read files in your project
- Edit or create files
- Run terminal commands
- Inspect output and revise its work
- Ask for permission before important actions

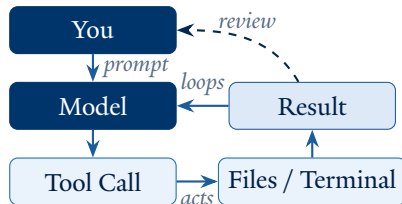
You are still responsible for the goal, judgment, and final review.

Chat Assistant vs. Coding Agent

Chat Assistant



Claude Code



Claude can act, inspect results, and iterate

Claude Desktop is useful for conversation. Claude Code is useful when the work lives in files.

Working Location Matters

Claude can only work with the location you give it.

Place	What Claude sees
Claude Desktop	Files you attach or upload
Terminal	The folder where you run <code>claude</code>
VS Code extension	The folder opened in VS Code
Positron extension	The folder opened in Positron

For this workshop, I will use the Claude Code extension inside Positron.

The Agentic Loop

Claude Code usually works in a loop:

1. You describe the task.
2. Claude gathers context.
3. Claude proposes or takes an action.
4. Claude reads the result.
5. Claude either stops or keeps iterating.

You can interrupt, redirect, or add information at any point.

A Good Prompt Has Four Parts

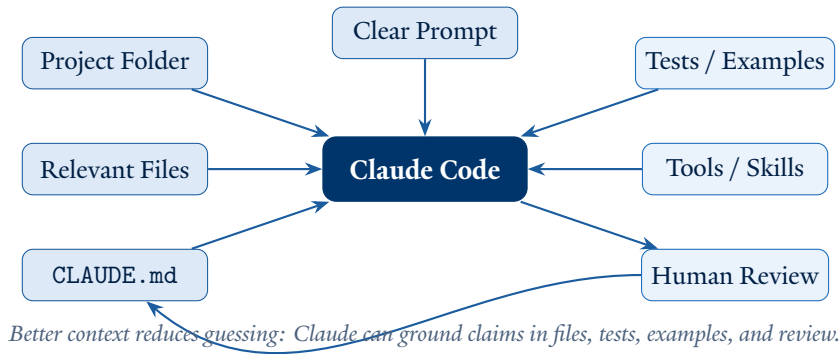
Part	Question to answer
Task	What do you want done?
Context	Which files, folders, or variables matter?
Output	What should Claude produce?
Done	How should Claude verify the result?

Write prompts the way you would brief a capable collaborator who is new to the project.

Weak Prompt, Strong Prompt

Weak	Strong
Analyze this data.	Read <code>data/input.csv</code> , summarize missingness by column, and save the result to <code>output/missingness.csv</code> .
Fix my script.	In <code>analysis/run_model.R</code> , find why the script fails, explain the issue, fix it, and rerun the script.
Make a plot.	Create a plot from <code>results/summary.csv</code> , save it to <code>figures/summary.png</code> , and report where the file was written.

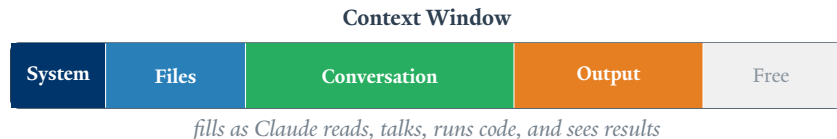
Context Engineering is the New Prompt Engineering



Hallucinations are more likely when Claude has to infer missing context. Good context gives it evidence to inspect and checks to run.

Context Window

Claude has a working memory for each session.



What fills context:

- Prompts and responses
- Files Claude reads
- Commands Claude runs
- Tool outputs and error messages
- Plans, edits, and verification results

Tools

Tools are how Claude Code acts on your project.

Tool type	Example
Read files	Inspect scripts, data dictionaries, configs
Edit files	Modify code, write documentation, create outputs
Run commands	Execute R, Python, shell, tests, render steps
Search context	Find files, functions, errors, or patterns
Track tasks	Keep multi-step work organized

Claude chooses tools based on your prompt and the available context.

CLAUDE.md

CLAUDE.md is a project instruction file.

Place it at the root of your project:

```
my-project/  
  CLAUDE.md  
  data/  
  scripts/  
  output/
```

Claude Code reads it automatically when it starts in that project folder.

What Goes in CLAUDE.md

Good things to include:

- Project structure
- How to run the project
- Preferred packages or commands
- Output folders
- Naming conventions
- Common mistakes Claude should avoid

Keep it short. This is onboarding for the project, not a full manual.

CLAUDE.md Example

Project Instructions

Use R scripts in `scripts/` for analysis work.

Save generated tables to `output/tables/`.

Save generated figures to `output/figures/`.

Before finishing a task:

- Run the script you changed.
- Report any warnings or errors.
- Do not overwrite raw data files.

Start without one if you are unsure. Add instructions when you find yourself repeating the same correction.

Skills

A skill is a reusable procedure Claude can load when a task matches.

Skills are useful for:

- A repeated analysis or reporting workflow
- A required output format
- A review checklist
- A team convention
- A verification procedure

The idea is simple: write the procedure once, reuse it across projects.

Subagents

A subagent works in a separate context window.

Good uses:

- Review code that the main session just wrote
- Search a project for a specific pattern
- Investigate a narrow question in parallel
- Compare two possible approaches

For review tasks, give the subagent read-only instructions: flag issues, do not silently fix them.

Other Power-Ups

You do not need these on day one, but they are useful later.

Feature	Use it for
MCP servers	Connecting Claude to external tools
Hooks	Deterministic checks before or after actions
Custom commands	Repeated project actions

The core workflow comes first: prompt, inspect, act, verify.

Why Compaction Can Hurt

When context gets full, Claude compacts the session: detailed history is replaced by a shorter summary.

After Compaction



older file details, decisions, and command outputs may be compressed or lost

Earlier file details, intermediate decisions, and command outputs can be compressed or lost entirely.

The Compaction Trade-off

Performance Heuristic

$$\text{Performance} \approx \frac{\text{correctness}^2 \times \text{completeness}}{\text{size}}$$

Compaction reduces **size** — but it also risks reducing **correctness** and **completeness**. The numerator typically suffers more than the denominator gains.

The fix is not to avoid compaction, but to manage it intentionally:

- Compact **before** the window is full, not after
- Write key decisions and outputs to files **first** so they survive the summary
- Do not wait for auto-compaction — trigger it yourself with `/compact`

Context Commands and Habits

Command	Use it when...
<code>/context</code>	Check how much working memory remains
<code>/compact</code>	Mid-task and running low – compact and continue
<code>/clear</code>	Starting a new task – full clean slate

- Use one session for one main task
- Save outputs to files; write progress to `PLAN.md` – both survive `/clear`
- Put reusable project facts in `CLAUDE.md`

Permission Modes

Use `Shift + Tab` to cycle modes.

Mode	What happens	Good for
Approval	Claude asks before edits and commands	New projects, risky changes
Auto-accept	File edits are accepted automatically	Fast iteration
Plan Mode	Claude reads and plans before changing files	Multi-step work

Start conservative. Speed up only when the task is low risk.

Plan Mode

Plan Mode is the best place to course-correct.

Use it when:

- The task touches multiple files.
- You are not sure where the change belongs.
- You want Claude to explain the approach before acting.
- The cost of a wrong edit would be annoying to unwind.

Cycle into Plan Mode with `Shift + Tab`.

Choosing a Claude Model

Claude Code supports model aliases, so you do not need to remember exact model IDs.

Alias	Use it for
haiku	Fast, simple tasks: summaries, small edits, routine lookups
sonnet	Everyday coding, analysis scripts, debugging, documentation
opus	Hard planning, ambiguous tasks, architecture, deep review

Use `/model` during a session or start Claude with `claude --model sonnet`.

Do Not Use It as a Flotation Device

Claude Code is helpful, but it is not a substitute for understanding the work.

Good fits:

- Tedious file edits and project navigation
- Drafting scripts from clear requirements
- Debugging with visible error messages
- Creating repeatable checks or reports

Weaker fits:

- Vague goals with no success criterion
- Work you cannot evaluate yourself
- High-stakes changes without tests or review
- One-shot “make this correct” requests

Build Trust with Benchmarks

Whether agentic AI is an appropriate tool is an *empirical question*:

1. Start with a small task where you know the expected result.
2. Run the same task manually or with an existing script.
3. Compare human and AI outputs.
4. Turn the comparison into a test, check, or verification skill.
5. Expand the workflow only after it passes benchmarks.

The goal is an improving toolchain, not a single impressive answer.

Review Before You Trust

Pause at natural checkpoints:

- Did Claude use the right folder?
- Did it read the right files?
- Did it understand the task?
- Did it change only what you expected?
- Did it run the relevant checks?
- Can you explain the result yourself?

The useful workflow is not autopilot. It is delegation with review.

A Reliable Workflow

Use the same rhythm for most Claude Code work:

Step	What you do
Explore	Ask Claude to inspect the project and summarize what it sees
Plan	Use Plan Mode for multi-step work
Act	Let Claude make the smallest useful change
Verify	Run the relevant command, test, render, or check
Save	Keep useful outputs in files, not only in chat

Demo Roadmap

In the demo, we will practice the workflow on a real project folder.

Demo step	What to watch for
Open folder	Claude sees the right project location
First prompt	Task, files, output, done condition
Plan or inspect	Claude gathers context before acting
Run work	Claude edits or writes files
Verify	Claude checks outputs and reports results

Demo 1: Merge and Count

The first demo is a follow-along task.

We will ask Claude to:

1. Inspect two data files.
2. Identify the common key.
3. Merge the files.
4. Recode a few columns.
5. Save a summary table.

The point is not the dataset. The point is the workflow: prompt, inspect, act, verify.

Demo 1 Prompt Pattern

Read [file 1] and [file 2].

Merge them on [key].

Create [summary output].

Save the result to [output path].

Before finishing, report any unmatched rows
and confirm the file was written.

Follow-Along Checkpoints

During the demo, pause and check:

1. What folder is Claude working in?
2. What files did Claude inspect?
3. What command did Claude run?
4. What file did Claude create or modify?
5. What evidence shows the task is done?

These same questions work for almost any Claude Code task.

Demo 2: Verification Pattern

The second demo shows a verification habit.

We will ask Claude to:

1. Write or run an analysis.
2. Identify the key outputs.
3. Reproduce the result another way.
4. Compare the outputs.
5. Report whether they match.

This is where skills or subagents become useful: they make review repeatable.

Demo 2: Cross-Language Verification

One useful verification pattern is to reproduce an analysis in a second language.

The idea:

1. Run the original analysis.
2. Identify the key numerical outputs.
3. Recreate the analysis in another language.
4. Compare the outputs.
5. Flag any mismatch for human review.

A match is not a proof, but it is a strong check against transcription errors, package defaults, and silent mistakes.

Demo 2: Skill File

A skill is a Markdown file with YAML frontmatter.

```
---  
name: cross-lang-verify  
description: Verify analysis results by replicating in a second  
            language and comparing numerical outputs.  
model: claude-opus-4-7  
persona: |  
    You are a rigorous statistical programmer conducting a  
    formal verification audit. Replicate faithfully and flag  
    discrepancies rather than explain them away.  
---
```

The description tells Claude when the skill is relevant.

Demo 2: Skill Procedure

The body of the skill gives Claude the steps to follow.

```
## Procedure
1. Identify the primary outputs to verify
2. Choose the target language
   - R analysis -> Python replication
   - Python analysis -> R replication
3. Write the replication to verify_<script>.<ext>
4. Run both versions
5. Compare key outputs to 4 decimal places
6. Report VERIFIED or MISMATCH with exact values
```

This turns a review habit into a reusable workflow.

Demo 2: Where the Skill Lives

A skill is just a file in your project folder.

```
project-folder/  
  .claude/  
    skills/  
      cross-lang-verify.md
```

In Positron, you can show this in the file explorer:

1. Open the project folder.
2. Create a `.claude` folder if it does not exist.
3. Inside it, create a `skills` folder.
4. Save the skill as `cross-lang-verify.md`.

The dot in `.claude` means “project configuration folder.”

Demo 2: Let Claude Create It

If you are terminal-shy, ask Claude Code to create the file for you.

```
Create a skill file at .claude/skills/cross-lang-verify.md  
for cross-language verification.
```

```
Use the procedure from the open cross-lang-verify.md file.
```

Then review the file in the Positron file explorer before using it.

The important habit: Claude can create project files, but you still inspect what it wrote.

Demo 2: Using the Skill

Once the file exists, invoke it directly:

```
Use the cross-lang-verify skill to verify this analysis.
```

Or ask naturally:

```
Verify this analysis by reproducing the key outputs in a second language.
```

Claude uses the skill because the request matches the skill description.

Demo 2: What We Will Ask

In the live demo, Claude will:

1. Run or inspect the original analysis.
2. Record the key outputs.
3. Write a second-language replication.
4. Compare the results.
5. Report whether the outputs match.

Useful questions to bring back to your own work:

1. What is a low-risk task I could try first?
2. What files would Claude need to see?
3. What output would count as done?
4. What checks would I require before trusting it?

Claude Code is most useful when you combine clear delegation with careful review.

Appendix: Quick Reference

Action	How
Start Claude Code	<code>claude</code>
Change model	<code>/model</code> or <code>claude --model sonnet</code>
Cycle modes	<code>Shift + Tab</code>
Check context	<code>/context</code>
Compact and continue	<code>/compact</code>
Start fresh	<code>/clear</code>
Get help	<code>/help</code>
Exit	<code>/exit</code>

Use this as a memory aid, not as a list to memorize.

Appendix: Customization Reference

Feature	Where it lives	Use it for
CLAUDE.md	Project root	Project instructions Claude reads automatically
Skills	.claude/skills/	Reusable procedures
Subagents	Managed with /agents	Focused review or parallel investigation
MCP servers	Managed with /mcp	Connections to external tools
Hooks	Managed with /hooks	Deterministic commands at lifecycle events

Start with CLAUDE.md. Add the rest only when there is a repeated workflow worth formalizing.

Appendix: Claude Code and Codex

Both are coding agents. Use the one that is easiest to verify for the task.

Task	Claude Code	Codex
Live teaching or workshop demo	Strong fit in terminal/editor	Strong fit if already using Codex workflow
Local project exploration	Strong fit in an opened folder	Strong fit with CLI or IDE workflows
Reusable project instructions	CLAUDE.md, skills, subagents	Project instructions and review workflows
Independent verification	Useful as one checker	Useful as another checker

Best practice: compare outputs across tools, tests, or human review instead of trusting any single agent.